

Midterm-Part 1

Hands-On Activity

CNN-LSTM Hybrid Models and Hyperparameter Optimization

Companion activity for 04_cnn_lstm_energy_forecasting.ipynb and 05_cnn_lstm_optimized_energy_forecasting.ipynb (extends the RNN/LSTM/GRU set, same dataset and task)

Estimated time: 140 minutes **Prerequisite:** a completed run of 04_cnn_lstm_energy_forecasting.ipynb and 05_cnn_lstm_optimized_energy_forecasting.ipynb (and, ideally, the earlier RNN/LSTM/GRU set for Part D).

Learning Objectives

- Explain why combining Conv1D and LSTM layers can help a forecasting model, and what causal padding does in a temporal Conv1D layer.
- Build and interpret a CNN-LSTM Sequential model's shape flow (Input → Conv1D → MaxPooling1D → LSTM → Dense).
- Define a Keras Tuner search space and run a random search over CNN-LSTM hyperparameters.
- Compare a hand-picked baseline against a hyperparameter-tuned model using validation loss and parameter count, and connect search results back to which hyperparameters mattered most.
- Distinguish MISO (multivariate input, univariate output) from MIMO (multivariate input, multivariate output) configurations, and modify a Sequential model's target construction and final Dense layer to switch between them.
- Place the plain LSTM, baseline CNN-LSTM, and tuned CNN-LSTM results into one coherent comparison on the same forecasting task.

Materials

- 01_vanilla_rnn_energy_forecasting.ipynb, 02_lstm_energy_forecasting.ipynb, 03_gru_energy_forecasting.ipynb (for the Part D comparison).
- 04_cnn_lstm_energy_forecasting.ipynb and 05_cnn_lstm_optimized_energy_forecasting.ipynb (this activity's primary notebooks).
- A Python environment with TensorFlow/Keras, keras-tuner, pandas, matplotlib, pydot, and Graphviz.
- Optional: the real Kaggle dataset. Works either way — note in your answers which data source you used.

Part A — Conceptual Warm-Up (individual, ~15 min)

Answer briefly (2–4 sentences each) before touching any code.

A1.

Notebook 04's Conv1D layer uses padding="causal". Explain what this padding choice does to which timesteps each Conv1D output position can "see", and why this matters more for architectures that produce one prediction per timestep (like the synced many-to-many notebook) than it does for this specific many-to-one setup, where the label is already outside the whole window.

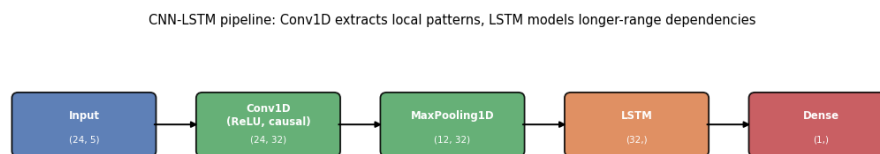


Figure A1. The CNN-LSTM pipeline in `04_cnn_lstm_energy_forecasting.ipynb`: shapes shown are for the default `FILTERS=32`, `WINDOW=24`, `UNITS=32`.

Answer:

Causal padding ensures that each Conv1d output at time t can only see the current and previous timesteps, not future timesteps. It's important for many-to-many forecasting, for example, because it avoids future information in later timesteps "leaking" into the earlier timesteps' predictions. Not that it's a huge issue for many-to-one, since the label is outside the entire window.

A2.

MaxPooling1D(pool_size=2) shrinks the sequence from 24 timesteps to 12 before the LSTM sees it. What information is discarded by pooling, and why might discarding it still help the LSTM that follows?

Answer:

MaxPooling1D reduces the sequence from 24 timesteps to 12 timesteps, by taking the maximum of two timesteps. This will result in a loss of some information, but might also cause faster learning since the LSTM has less input to process. MaxPooling1D can also help the LSTM to learn stronger local patterns.

A3.

In notebook 05, `build_model(hp)` calls `model.compile(...)` inside the function itself, and the notebook later uses `fit_only()` rather than `compile_and_fit()` to train the best model. Why is this necessary here, when every earlier notebook used `compile_and_fit()` without issue?

Answer:

In Notebook 05, why must we compile the model within `build_model(hp)`? Because Keras Tuner will create and evaluate many different models as it searches for the best hyperparameters — each one must be compiled before training, hence `fit_only()` will train the best model, already compiled. The previous notebooks could make use of `compile_and_fit()` because they were not created and evaluated by the tuner

A4.

In one or two sentences each, distinguish grid search, random search, and Hyperband/Bayesian search as hyperparameter search strategies.

Answer:

- Grid Search: It is an exhaustive search in which all possible combinations of the selected hyperparameters are tested. It can be computationally intensive.
- Random Search: It randomly samples the hyperparameter space and evaluates each randomly chosen set of hyperparameters. It can be more efficient than grid search, especially for some high-dimensional spaces, when only a few hyperparameters influence the results.
- Hyperband/Bayesian Search: The first algorithm, Hyperband, terminates poorly performing training jobs early and focuses on the most promising regions of the hyperparameter space by using a large budget for training the selected machine learning (ML) models. The second algorithm, Bayesian search, uses information from previous searches to select the most promising combination of hyperparameters to evaluate.

Name: Bernard G. Yasay

Doctor of Engineering with Specialization in Computer Engineering

DOES 1C3-DIT/DENG11G1 – Bioinformatics

Part B — Hands-On Modification Tasks (pair or individual, ~45 min)

Do each task in the notebook named. Re-run from the point of change onward, then record what changed using the printed “ran X/100 epochs ... params=...” line (or the tuner's results_summary for B3–B5).

Task B1 — Deepen the conv stack (Notebook 04)

Uncomment the second Conv1D + MaxPooling1D block in the baseline model.

```
[14]: model_cnn_lstm = tf.keras.Sequential([
    tf.keras.layers.Input(shape=(WINDOW, N_FEATURES)),
    tf.keras.layers.Conv1D(filters=FILTERS, kernel_size=KERNEL_SIZE, activation="relu", padding="causal"),
    tf.keras.layers.MaxPooling1D(pool_size=POOL_SIZE),

    # --- Stack an additional Conv1D block before the LSTM: -----
    tf.keras.layers.Conv1D(filters=FILTERS, kernel_size=KERNEL_SIZE, activation="relu", padding="causal"),
    tf.keras.layers.MaxPooling1D(pool_size=POOL_SIZE),
    # -----
    tf.keras.layers.LSTM(UNITS, return_sequences=False),
    tf.keras.layers.Dense(1),
], name="cnn_lstm_many_to_one")

model_cnn_lstm.summary()
show_model_diagram(model_cnn_lstm, "cnn_lstm_baseline.png")
```

Record: new parameter count, new val_loss/val_mae.

cnn_lstm_baseline: ran 100/100 epochs final val_loss=0.00839 val_mae=0.07301 params=11,969

Reflect: Did a deeper conv stack help on this dataset size, or was one Conv1D block enough?

The deeper model added 3,104 parameters (~35% more), but the validation results were essentially unchanged.

Val loss: 0.00825 → 0.00839, slightly worse

Val MAE: 0.07306 → 0.07301, only 0.00005 better

Parameters: 8,865 → 11,969, substantially more complex

So, the additional Conv1D block did not provide a meaningful improvement. The tiny MAE improvement is too small to justify the extra model complexity, especially since the validation loss actually increased.

Name: Bernard G. Yasay

Doctor of Engineering with Specialization in Computer Engineering

DOES 1C3-DIT/DENG11G1 – Bioinformatics

Task B2 — Change kernel and pool size (Notebook 04)

Change KERNEL_SIZE from 3 to 5, and POOL_SIZE from 2 to 3. Re-run and note the new shapes at each pipeline stage (update the draw_pipeline() call's shape labels to match, or just compute them by hand).

```
[10]: def make_many_to_one(data: pd.DataFrame, window: int = 24):
    """window hours of history -> the next single target value."""
    values = data.values
    target = data[TARGET_COL].values
    X, y = [], []
    for t in range(len(values) - window):
        X.append(values[t : t + window])
        y.append([target[t + window]])
    return np.array(X), np.array(y)

WINDOW = 24
UNITS = 32      # LSTM units, kept equal to the earlier LSTM notebook for a fair comparison
FILTERS = 32    # Conv1D filters
KERNEL_SIZE = 5
POOL_SIZE = 3
EPOCHS = 100
BATCH_SIZE = 64

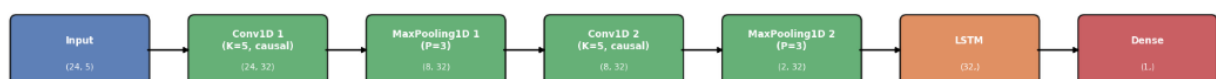
Xm1, ym1 = make_many_to_one(train_scaled, window=WINDOW)
Xv_m1, yv_m1 = make_many_to_one(val_scaled, window=WINDOW)
print("X shape:", Xm1.shape, " y shape:", ym1.shape)

X shape: (2076, 24, 5)  y shape: (2076, 1)
```

1. Input: (24, 5)
2. Conv1D 1: (24, 32) ; Causal padding preserves the 24 time steps.
3. MaxPooling1D 1: (8, 32) ; $|24 \div 3| = 8$
4. Conv1D 2: (8, 32) ; Causal padding preserves the 8 time steps.
5. MaxPooling1D 2: (2, 32) ; $|8 \div 3| = 2$
6. LSTM: (32,)
7. Dense: (1,)

```
[14]: draw_pipeline(
    stages=[
        ("Input", "(24, 5)", "#4C72B8"),
        ("Conv1D 1\n(K=5, causal)", "(24, 32)", "#55A868"),
        ("MaxPooling1D 1\n(P=3)", "(8, 32)", "#55A868"),
        ("Conv1D 2\n(K=5, causal)", "(8, 32)", "#55A868"),
        ("MaxPooling1D 2\n(P=3)", "(2, 32)", "#55A868"),
        ("LSTM", "(32,)", "#DD8452"),
        ("Dense", "(1,)", "#C44E52"),
    ],
    title="CNN-LSTM pipeline: Kernel=5, Pool=3",
    figsize=(16, 3)
)
```

CNN-LSTM pipeline: Kernel=5, Pool=3



Name: Bernard G. Yasay
Doctor of Engineering with Specialization in Computer Engineering
DOES 1C3-DIT/DENG11G1 – Bioinformatics

Record: the new post-pooling sequence length, and the new val_loss.

post-pooling sequence length = 2 (based on the above computation), val_loss=0.00899

Task B3 — Widen the search (Notebook 05)

Increase MAX_TRIALS from 8 to 20 and SEARCH_EPOCHS from 20 to 30. Re-run the search cell.

```
[ ]: MAX_TRIALS = 20
      SEARCH_EPOCHS = 30

tuner = kt.RandomSearch(
    build_model,
    objective="val_loss",
    max_trials=MAX_TRIALS,
    executions_per_trial=1,
    overwrite=True,
    directory="kt_search",
    project_name="cnn_lstm_energy",
    seed=42,
)

tuner.search(
    X_m1, y_m1,
    validation_data=(Xv_m1, yv_m1),
    epochs=SEARCH_EPOCHS,
    batch_size=BATCH_SIZE,
    verbose=0,
)

tuner.results_summary(num_trials=3)
```

Record: the new best hyperparameters and best val_loss, and how much longer the search took.

Best hyperparameters found:

filters: 32

kernel_size: 3

pool_size: 3

lstm_units: 48

dropout_rate: 0.2

learning_rate: 0.0006752863927347823

tuned: ran 100/100 epochs final val_loss=0.00825 val_mae=0.07264 params=16,113

Reflect: Was the improvement worth the extra compute time?

With MAX_TRIALS = 8 and SEARCH_EPOCHS = 20, the best hyperparameters appeared at around 168 seconds.

Best hyperparameters found:

Name: Bernard G. Yasay
Doctor of Engineering with Specialization in Computer Engineering
DOES 1C3-DIT/DENG11G1 – Bioinformatics

filters: 48

kernel_size: 2

pool_size: 2

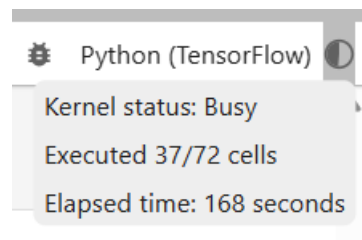
lstm_units: 32

dropout_rate: 0.4

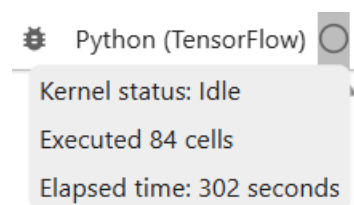
learning_rate: 0.001715074355925934

The tuned model ran for 100/100 epochs and achieved a final val_loss = 0.00831 and val_mae = 0.07284, with 10,929 parameters.

However, only 37 out of the 72 cells had been executed, and the remaining cells were taking much longer to finish. Although all the output graphs in the program had already appeared, I decided to stop the running kernel because the execution was taking too long.



With MAX_TRIALS = 20 and SEARCH_EPOCHS = 30, the best hyperparameters appeared at around 302 seconds.



The increase from MAX_TRIALS = 8 and SEARCH_EPOCHS = 20 to MAX_TRIALS = 20 and SEARCH_EPOCHS = 30 resulted in only a marginal improvement in model performance. The validation loss decreased from 0.00831 to 0.00825, while the validation MAE decreased from 0.07284 to 0.07264. However, the computation time increased from approximately 168 seconds to 302 seconds, or about 80% longer. The number of parameters also increased from 10,929 to 16,113. Therefore, the additional hyperparameter search was not substantially worth the extra computation time because the improvement in validation performance was very small. The first configuration may be preferred because it provides nearly the same performance while requiring significantly less computational time and producing a smaller model.

Task B4 — Try a different search strategy (Notebook 05)

Replace `kt.RandomSearch(...)` with `kt.Hyperband(build_model, objective="val_loss", max_epochs=SEARCH_EPOCHS, factor=3, directory="kt_search", project_name="cnn_lstm_hyperband", overwrite=True, seed=42)`, then call `tuner.search(...)` the same way.

Name: Bernard G. Yasay

Doctor of Engineering with Specialization in Computer Engineering

DOES 1C3-DIT/DENG11G1 – Bioinformatics

```
MAX_TRIALS = 20
SEARCH_EPOCHS = 30

tuner = kt.Hyperband(build_model,
                     objective="val_loss",
                     max_epochs=SEARCH_EPOCHS,
                     factor=3, directory="kt_search",
                     project_name="cnn_lstm_hyperband",
                     overwrite=True,
                     seed=42,
                     )

tuner.search(
    Xm1, ym1,
    validation_data=(Xv_m1, yv_m1),
    epochs=SEARCH_EPOCHS,
    batch_size=BATCH_SIZE,
    verbose=0,
)

tuner.results_summary(num_trials=3)
```

Record: Hyperband's best val_loss and total search time, compared to RandomSearch's.

Best hyperparameters found:

filters: 48

kernel_size: 3

pool_size: 2

lstm_units: 96

dropout_rate: 0.30000000000000004

learning_rate: 0.004569579155191588

tuner/epochs: 30

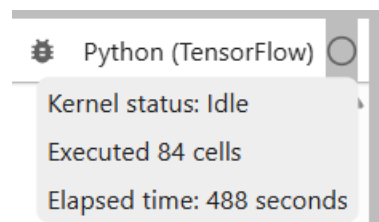
tuner/initial_epoch: 10

tuner/bracket: 2

tuner/round: 2

tuner/trial_id: 0067

tuned: ran 100/100 epochs final val_loss=0.00870 val_mae=0.07387 params=56,545



Name: Bernard G. Yasay

Doctor of Engineering with Specialization in Computer Engineering

DOES 1C3-DIT/DENG11G1 – Bioinformatics

Task B5 — Add a hyperparameter (Notebook 05)

Add one more tunable hyperparameter to `build_model(hp)` — for example an L2 regularization strength on the Dense layer, or a second Dense layer's width. Re-run the search.

```
[ ]: def build_model(hp):  
    filters = hp.Int("filters", min_value=16, max_value=64, step=16)  
    kernel_size = hp.Choice("kernel_size", [2, 3, 5])  
    pool_size = hp.Choice("pool_size", [2, 3])  
    lstm_units = hp.Int("lstm_units", min_value=16, max_value=96, step=16)  
    dropout_rate = hp.Float("dropout_rate", min_value=0.0, max_value=0.4, step=0.1)  
    learning_rate = hp.Float("learning_rate", min_value=1e-4, max_value=1e-2, sampling="log")  
  
    # NEW TUNABLE HYPERPARAMETER  
    l2_strength = hp.Float("l2_strength", min_value=1e-6, max_value=1e-2, sampling="log")  
  
    model = tf.keras.Sequential([  
        tf.keras.layers.Input(shape=(WINDOW, N_FEATURES)),  
        tf.keras.layers.Conv1D(filters=filters, kernel_size=kernel_size, activation="relu", padding="causal"),  
        tf.keras.layers.MaxPooling1D(pool_size=pool_size),  
        tf.keras.layers.LSTM(lstm_units, return_sequences=False),  
        tf.keras.layers.Dropout(dropout_rate),  
        #tf.keras.layers.Dense(1),  
        tf.keras.layers.Dense(1, kernel_regularizer=tf.keras.regularizers.l2(l2_strength)),  
    ])  
    model.compile(  
        optimizer=tf.keras.optimizers.Adam(learning_rate=learning_rate),  
        loss="mse",  
        metrics=["mae"],  
    )  
    return model  
  
print("Search space: filters in [16,32,48,64], kernel_size in {2,3,5}, pool_size in {2,3}, "  
      "lstm_units in [16..96 step 16], dropout_rate in [0.0..0.4], learning_rate in [1e-4, 1e-2] (log scale),l2_strength in [1e-6,1e-2] (log scale)")
```

Record: whether your new hyperparameter appears with a consistent value across the top 3 trials in `tuner.results_summary(num_trials=3)`.

Results summary

Results in `kt_search\cnn_lstm_hyperband`

Showing 3 best trials

Objective(name="val_loss", direction="min")

Trial 0050 summary

Hyperparameters:

filters: 64

kernel_size: 2

pool_size: 3

lstm_units: 80

dropout_rate: 0.1

learning_rate: 0.0036110447055948225

l2_strength: 2.1301425578360572e-05

tuner/epochs: 30

tuner/initial_epoch: 10

tuner/bracket: 3

tuner/round: 3

tuner/trial_id: 0046

Score: 0.007943629287183285

Name: Bernard G. Yasay

Doctor of Engineering with Specialization in Computer Engineering

DOES 1C3-DIT/DENG11G1 – Bioinformatics

Trial 0051 summary

Hyperparameters:

filters: 16

kernel_size: 2

pool_size: 3

lstm_units: 48

dropout_rate: 0.4

learning_rate: 0.0065117874801730825

l2_strength: 4.3829958444048034e-05

tuner/epochs: 30

tuner/initial_epoch: 10

tuner/bracket: 3

tuner/round: 3

tuner/trial_id: 0047

Score: 0.007945679128170013

Trial 0072 summary

Hyperparameters:

filters: 64

kernel_size: 5

pool_size: 3

lstm_units: 32

dropout_rate: 0.2

learning_rate: 0.0070723782496249854

l2_strength: 3.7861707086815074e-06

tuner/epochs: 30

tuner/initial_epoch: 10

tuner/bracket: 2

tuner/round: 2

tuner/trial_id: 0067

Score: 0.008049007505178452

Part C — Extending to Multivariate Input and Output (individual or pair, ~50 min)

Every model built so far in Notebooks 04 and 05 takes multivariate input (the 5-column `FEATURE_COLS` window) and produces univariate output (a single predicted price). This input/output combination is sometimes labeled MISO (Multiple Inputs, Single Output). This part has you build the MIMO (Multiple Inputs, Multiple Outputs) counterpart: the same 5-feature input window, but predicting two targets at once, still on the same 24-hour many-to-one task.

C1 — Name the baseline (conceptual, ~5 min)

Label the baseline CNN-LSTM model from Notebook 04 using the MISO/MIMO terminology, and state precisely what “multivariate” refers to on the input side (`FEATURE_COLS`) versus what would have to change on the output side to make the model MIMO instead of MISO.

Based on the definition, the `cnn_lstm_many_to_one` model is a Multiple Input Single Output (MISO) model.

Multivariate Input: The input to the model (`WINDOW`, `N_FEATURES`) = (24, 5) indicates that for a prediction of the next hour's price actual, the model takes 24 hours (`WINDOW`) of 5 features (total load actual, generation solar, generation wind onshore, generation fossil gas, price actual) as input. The 'multivariate' aspect on the input side refers to the fact that the model uses several distinct features at each timestep in its input sequence.

Single Output: The final `tf.keras.layers.Dense(1)` layer has only one unit, which outputs the prediction for the next hour's price actual, hence single output.

To turn this model from MISO (Multiple Input Single Output) to MIMO (Multiple Input Multiple Output) you would need to make changes to the output layer in order to predict multiple values.

Some options could be:

1. Predict multiple time steps into the future of the target variable: For example, instead of predicting just the price actual of the next hour, predict the price actual of the next 3 hours. In this case you would need to change from `tf.keras.layers.Dense(1)` to `tf.keras.layers.Dense(3)` if in the LSTM `return_sequences=False`. Else, you might have to make some changes in the heads of the model according to the sequence prediction task.
2. Predict multiple target variables: For example, instead of predicting just the price actual of the next hour, predict the price actual and the total load actual. In this case you would need to change from `tf.keras.layers.Dense(1)` to `tf.keras.layers.Dense(N_TARGETS)` where `N_TARGETS` is the number of variables you want to predict and also change the `y` labels accordingly.

Name: Bernard G. Yasay

Doctor of Engineering with Specialization in Computer Engineering

DOES 1C3-DIT/DENG11G1 – Bioinformatics

C2 — Build a MIMO variant (hands-on, ~30 min)

In a copy of Notebook 04's baseline model, build a MIMO version that predicts two targets at once instead of one:

1. Define TARGET_COLS = ["price actual", "total load actual"] alongside the existing single TARGET_COL.

```
[5]: import os
import numpy as np
import pandas as pd

DATA_DIR = "data"
ENERGY_CSV = os.path.join(DATA_DIR, "energy_dataset.csv")

FEATURE_COLS = [
    "total load actual",
    "generation solar",
    "generation wind onshore",
    "generation fossil gas",
    "price actual",
]
TARGET_COL = "price actual"

#1 DEFINE TARGET COLUMNS
TARGET_COLS = ["price actual", "total load actual"]
```

2. Write a make_many_to_one_multi(data, window=24) that mirrors make_many_to_one, but builds y from both target columns: y.append(data[TARGET_COLS].values[t + window]) so each label is a 2-vector instead of a scalar.

```
[10]: #2 CREATE MANY-TO-ONE MULTI-TARGET WINDOWING FUNCTION

def make_many_to_one(data: pd.DataFrame, window: int = 24):
    """`window` hours of history -> the next values of both targets.
    Each y label is a 2-vector: [price actual, total load actual]"""

    values = data.values
    target = data[TARGET_COLS].values
    X, y = [], []
    for t in range(len(values) - window):
        X.append(values[t : t + window])
        y.append([target[t + window]])
    return np.array(X), np.array(y)

WINDOW = 24
UNITS = 32          # LSTM units, kept equal to the earlier LSTM notebook for a fair comparison
FILTERS = 32        # Conv1D filters
KERNEL_SIZE = 5
POOL_SIZE = 3
EPOCHS = 100
BATCH_SIZE = 64

Xm1, ym1 = make_many_to_one(train_scaled, window=WINDOW)
Xv_m1, yv_m1 = make_many_to_one(val_scaled, window=WINDOW)
print("X shape:", Xm1.shape, " y shape:", ym1.shape)
```

Name: Bernard G. Yasay

Doctor of Engineering with Specialization in Computer Engineering

DOES 1C3-DIT/DENG11G1 – Bioinformatics

3. Change the model's final layer from `tf.keras.layers.Dense(1)` to `tf.keras.layers.Dense(2)`.

```
•[15]: #3 Change the model's final layer from tf.keras.layers.Dense(1) to tf.keras.layers.Dense(2).

model_cnn_lstm = tf.keras.Sequential([
    tf.keras.layers.Input(shape=(WINDOW, N_FEATURES)),
    tf.keras.layers.Conv1D(filters=FILTERS, kernel_size=KERNEL_SIZE, activation="relu", padding="causal"),
    tf.keras.layers.MaxPooling1D(pool_size=POOL_SIZE),

    # --- Stack an additional Conv1D block before the LSTM: -----
    #tf.keras.layers.Conv1D(filters=FILTERS, kernel_size=KERNEL_SIZE, activation="relu", padding="causal"),
    #tf.keras.layers.MaxPooling1D(pool_size=POOL_SIZE),
    # -----
    tf.keras.layers.LSTM(UNITS, return_sequences=False),
    tf.keras.layers.Dense(2),
], name="cnn_lstm_many_to_many")

model_cnn_lstm.summary()
show_model_diagram(model_cnn_lstm, "cnn_lstm_baseline.png")

dense2_params = model_cnn_lstm.layers[-1].count_params()
print("Dense(2) parameter count:", dense2_params)
```

4. Retrain for the same EPOCHS = 100 and the same WINDOW, FILTERS, KERNEL_SIZE, POOL_SIZE, UNITS as the baseline.

```
#4 Retrain for the same EPOCHS = 100
import tensorflow as tf
import matplotlib.pyplot as plt
import matplotlib.patches as mpatches
from matplotlib.patches import FancyBboxPatch

tf.random.set_seed(42)
np.random.seed(42)

history_log = {}

_TARGET_MIN = scaler.data_min_[TARGET_IDX]
_TARGET_MAX = scaler.data_max_[TARGET_IDX]
```

```
#4 same WINDOW, FILTERS, KERNEL_SIZE, POOL_SIZE, UNITS as the baseline.
WINDOW = 24
UNITS = 32          # LSTM units, kept equal to the earlier LSTM notebook for a fair comparison
FILTERS = 32        # Conv1D filters
KERNEL_SIZE = 5
POOL_SIZE = 3
EPOCHS = 100
BATCH_SIZE = 64
```

Name: Bernard G. Yasay

Doctor of Engineering with Specialization in Computer Engineering

DOES 1C3-DIT/DENG11G1 – Bioinformatics

Record: the new y shape, the new Dense layer's parameter count, and the combined val_loss Keras reports (it averages MSE across both output units by default).

```
# 5. RECORD REQUIRED RESULTS
new_y_shape = ym1.shape
dense2_params = model_cnn_lstm.layers[-1].count_params()
combined_val_loss = hist_cnn_lstm.history["val_loss"][-1]

print("\n===== MIMO RESULTS =====")
print("New y shape:", new_y_shape)
print("Dense(2) parameter count:", dense2_params)
print("Combined final val_loss:", combined_val_loss)
print("Keras MSE is averaged across the two output units.")
```

===== MIMO RESULTS =====

New y shape: (2076, 2)

Dense(2) parameter count: 66

Combined final val_loss: 0.004971835762262344

Keras MSE is averaged across the two output units.

C3 — Per-target accuracy (hands-on/analysis, ~15 min)

The combined val_loss from C2 mixes both targets' errors together, and price and load live on very different numeric scales, so it does not tell you how well either target is predicted on its own. Inverse-transform each output column separately using that column's own scaler range, not TARGET_IDX alone — for example:

```
price_min, price_max = scaler.data_min_[FEATURE_COLS.index("price actual")],
scaler.data_max_[FEATURE_COLS.index("price actual")]

load_min, load_max = scaler.data_min_[FEATURE_COLS.index("total load actual")],
scaler.data_max_[FEATURE_COLS.index("total load actual")]
```

Then compute a separate MAE in real units (EUR/MWh for price, MW for load) for each of the two outputs.

Name: Bernard G. Yasay

Doctor of Engineering with Specialization in Computer Engineering

DOES 1C3-DIT/DENG11G1 – Bioinformatics

```
from sklearn.metrics import mean_absolute_error

# =====
# PER-TARGET VALIDATION MAE IN ORIGINAL UNITS
# =====

# Get each target's own position in FEATURE_COLS
price_idx = FEATURE_COLS.index("price actual")
load_idx = FEATURE_COLS.index("total load actual")

# Get each target's own MinMaxScaler range
price_min = scaler.data_min_[price_idx]
price_max = scaler.data_max_[price_idx]

load_min = scaler.data_min_[load_idx]
load_max = scaler.data_max_[load_idx]

# Separate scaled true/predicted values
price_true_scaled = yv_m1[:, 0]
price_pred_scaled = pred[:, 0]

load_true_scaled = yv_m1[:, 1]
load_pred_scaled = pred[:, 1]

# Inverse-transform each target using ITS OWN scaler range
price_true = price_true_scaled * (price_max - price_min) + price_min
price_pred = price_pred_scaled * (price_max - price_min) + price_min

load_true = load_true_scaled * (load_max - load_min) + load_min
load_pred = load_pred_scaled * (load_max - load_min) + load_min

# Calculate MAE in original units
price_mae = mean_absolute_error(price_true, price_pred)
load_mae = mean_absolute_error(load_true, load_pred)

print("\n===== PER-TARGET MIMO VALIDATION MAE =====")
print(f"Price MAE: {price_mae:.4f} EUR/MWh")
print(f"Load MAE : {load_mae:.4f} MW")

print("\nScaler ranges used:")
print(f"Price: {price_min:.4f} to {price_max:.4f} EUR/MWh")
print(f"Load : {load_min:.4f} to {load_max:.4f} MW")
```

Record: the per-target MAE for price and for load.

===== PER-TARGET MIMO VALIDATION MAE =====

Price MAE: 3.2322 EUR/MWh

Load MAE : 396.3865 MW

Scaler ranges used:

Price: 21.9120 to 66.7981 EUR/MWh

Load : 20148.0114 to 31801.3935 MW

Name: Bernard G. Yasay

Doctor of Engineering with Specialization in Computer Engineering

DOES 1C3-DIT/DENG11G1 – Bioinformatics

Reflect: Which target is predicted more accurately? Does training on both jointly seem to help or hurt the price prediction compared to the single-target (MISO) baseline's val_mae from Notebook 04?

The MIMO model achieved a validation MAE of 3.2322 EUR/MWh for Price and 396.3865 MW for Load. Due to the different units and scales of the two targets, comparing their raw MAE values is not entirely informative. In relative terms to the scaler range, the Price error is about 7.20%, while the Load error is about 3.40%, suggesting the prediction of the latter has a smaller relative error. To investigate if the joint MIMO training helped or harmed the Price prediction, the MIMO validation MAE for Price (3.2322 EUR/MWh) should be compared to the validation MAE of the single-target MISO model for Price from Notebook 04. A lower MAE would suggest an improvement, while a higher value would imply that jointly predicting Price and Load led to worse Price prediction than predicting it alone.

C4 — Reflection.

Does moving from MISO to MIMO change which typology diagram (one-to-one / many-to-one / one-to-many / many-to-many) applies to this architecture? Why or why not?

The transition from MISO to MIMO changes the input-output typology of the CNN-LSTM architecture, since the model transforms itself from a single target prediction to multiple ones. In the MISO scenario, the input to output is many-to-one, which means that multiple input features through time steps are processed to predict a singular target, such as the electricity price. In the MIMO scenario, the same multiple input features are processed through CNN and LSTM layers, but the output layer has two neurons, which means that both the electricity price and the total load are predicted, hence many-to-many multiple targets. Thus, it can be argued that the temporal structure of the CNN-LSTM has not changed, while the output layer was modified, since the single target in MISO is transformed to two targets in MIMO scenario. This transition is expressed in the experiment by replacing the last Dense(1) layer with a Dense(2) one. Therefore, the transition from MISO to MIMO scenario modified the model output from having one target to two, while the CNN-LSTM part remained the same.

Name: Bernard G. Yasay

Doctor of Engineering with Specialization in Computer Engineering

DOES 1C3-DIT/DENG11G1 – Bioinformatics

Part D — Cross-Model Comparison (individual or group, ~30 min)

Using the printed metrics from 02_lstm_energy_forecasting.ipynb (many_to_one), 04_cnn_lstm_energy_forecasting.ipynb (baseline), and 05_cnn_lstm_optimized_energy_forecasting.ipynb (tuned), fill in the table below. (The MIMO variant from Part C is left out of this table on purpose — see the note below it.)

Model	Params	val_loss	val_mae
Plain LSTM (many-to-one, Notebook 02)	4,897	0.00854	0.07293
CNN-LSTM baseline (Notebook 04)	14,337	0.00899	0.07538
CNN-LSTM tuned (Notebook 05)	8,865	0.00857	0.07383

Note: the MIMO model's val_loss from Part C is an average over two targets on different scales, so it is not on the same footing as the single-target val_loss values above — that is a real property of multi-output regression, not a mistake in your table.

Reflection questions

D1.

Between the plain LSTM and the CNN-LSTM baseline, which had the lower val_loss on your run? What does that suggest about how much position-independent local structure this dataset actually has for a convolution to exploit?

Answer:

The Plain LSTM had a lower validation loss than the CNN-LSTM baseline, with a val_loss of 0.00854 compared to 0.00899, a difference of 0.00045, or approximately 5% lower. This indicates that the Plain LSTM was performing slightly better on the validation set. This suggests that the dataset does not contain a significant amount of position-independent local structure that a convolutional layer would be able to exploit. The addition of the CNN component did not demonstrate any initial benefit compared to the LSTM, indicating that the temporal relationships captured by the LSTM proved to be more valuable than those extracted by the CNN.

D2.

Did hyperparameter tuning close the gap (or extend the lead) between the CNN-LSTM and the plain LSTM? State the direction and size of the change.

Answer:

The tuned CNN-LSTM had a validation loss of 0.00857. This was an improvement of 0.00042 over the baseline CNN-LSTM validation loss of 0.00899. This demonstrates that hyperparameter tuning was able to close most of the gap between the CNN-LSTM and the Plain LSTM. Compared with the Plain

Name: Bernard G. Yasay
Doctor of Engineering with Specialization in Computer Engineering
DOES 1C3-DIT/DENG11G1 – Bioinformatics

LSTM, which had a validation loss of 0.00854, the tuned CNN-LSTM only had a validation loss of 0.00003 higher, meaning that the two were fairly similar in terms of their performance. The same can be said for their validation MAE, with the tuned CNN-LSTM achieving 0.07383, compared with 0.07538 for the baseline CNN-LSTM, a reduction of 0.00155. Overall, hyperparameter tuning had substantially improved the performance of the CNN-LSTM and brought it much closer to the Plain LSTM.

D3.

Looking at your tuner's `results_summary()` across its top 3 trials, which single hyperparameter appeared most consistently favorable (e.g. did most top trials prefer a particular kernel size, a small or large dropout rate)?

Answer:

Looking at the top 3 trials, the most consistent hyperparameter is `pool_size = 3`. All three of the top trials selected `pool_size = 3`. Therefore, among the hyperparameters shown, pool size of 3 appeared most consistently favorable. The kernel size was less consistent: two trials preferred 2, while one preferred 5. Likewise, the dropout rate varied among 0.1, 0.4, and 0.2, so there was no clear consistent preference for a particular dropout value.

Deliverables

5. Your modified notebook(s), or a diff/screenshot, for each Part B task.
6. Your MIMO notebook variant from Part C, with the per-target MAE results.
7. Written answers to Parts A, C, and D.
8. The completed comparison table from Part D.